A

Data structures and Algorithm Micro Project Report on

MUSIC PLAYLIST MANAGER USING DOUBLY LINKED LIST IN C

Submitted in partial fulfilment of the requirements for the completion of the course

MICRO PROJECT

Submitted by

Ms.Rutuja Rakesh Hable	**1025091048**
Ms.Shreya Umesh Gadale	**1025091052**
Ms. Sakshi Ajay Patil	**1025091054**

Under the Guidance of

Dr. Asma A. Shaikh



DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

Sant Dnyaneshwar Shikshan Sanstha's

ANNASAHEB DANGECOLLEGE OFENGINEERING AND TECHNOLOGY, ASHTA

(An Empowered Autonomous Institute)

SHIVAJI UNIVERSITY, KOLHAPUR

Academic Year: 2026-267

Sant Dnyaneshwar Shikshan Sanstha's

ANNASAHEB DANGECOLLEGE OFENGINEERING AND TECHNOLOGY, ASHTA

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE



CERTIFICATE

This is to certify that the MiniProject report entitled " **MUSIC PLAYLIST MANAGER USING DOUBLY LINKED LIST IN C**" is a bonafide work carried out by the following student(s):

URN	Name of Student
1025091048	Rutuja Rakesh Hable
1025091052	Shreya Umesh Gadale
1025091054	Sakshi Ajay Patil

in partial fulfilment of the requirements for the completion of the Mini Project course [1ICEL318] **for the Second Year, Bachelor of Technology in DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATASCIENCE at Annasaheb Dange College of Engineering and Technology, Ashta**, for the **Academic Year 2026–27**.

Place: Ashta

Date: / /

Course coordinator

ACKNOWLEDGEMENT

We express our sincere gratitude to our Micro Project Guide, **Dr.Asma A. Shaikh**, for his invaluable guidance, constant support, andencouragement throughout the completion of our Mini Project work.

We are also deeply thankful to the Head of the Department, **Dr.Asma A. Shaikh** DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE, for their continuous guidance, valuable suggestions, and unwavering support, which greatly helped us in overcoming various challenges during the course of this Mini Project.

Our heartfelt thanks are extended to our Hon. Director, **Dr. L. Y. Waghmode**, for providing us with the necessary facilities and a motivating environment that enabled us to carry out our Mini Project work successfully. We would also like to express our appreciation to all the **faculty members, staff and management** of **Annasaheb Dange College of Engineering and Technology, Ashta**, for their cooperation and assistance throughout this journey.

Finally, we wish to convey our deep gratitude to our **friends and family members** for their constant encouragement, understanding, and moral support, which played a vital role in the successful completion of this Mini Project.

Rutuja Rakesh Hable

Shreya Umesh Gadale

Sakshi Ajay Patil

ABSTRACT

Abstract

Abstract should be straight to the point; not too descriptive but fully informative. Include the problem to be addressed, its importance/novelty, the approach to be adopted for solving the problem, highlighting novelty, if any. It should inform a reader whether to read or not the full text, and also give a precise idea about proposed work. It should be 200 to 250 words in length.

MUSIC PLAYLIST MANAGER USING DOUBLY LINKED LIST IN C

Data Structures and Algorithms Micro Project Report

This section contains the project-specific report content for the topic "**Music Playlist Manager Using Doubly Linked List in C**". The first five pages of the supplied template have been retained unchanged, as requested.

Department of Artificial Intelligence and Data Science Annasaheb Dange College of Engineering and Technology, Ashta.

Academic Year: 2026–27

TABLE OF CONTENTS

Section	Contents
1	Introduction
2	Problem Statement
3	Literature Survey
4	System Requirements
4.1	Hardware Requirements
4.2	Software Requirements
5	System Design
5.1	System Architecture
5.2	ER / Data Structure Diagram
5.3	Data Flow Diagram (DFD)
5.4	Use Case Diagram
5.5	Data Schema / Node Design
6	Implementation
6.1	Algorithm and Modular Design
6.2	C Source Code
6.3	SQL Queries, Tables and Normalization (Applicability Note)
6.4	Program Execution Snapshots
7	Results / Output
8	Applications
9	Advantages and Limitations
9.1	Advantages
9.2	Limitations
10	Conclusion
11	Future Scope
12	References
13	Appendix: Complexity Analysis and Test Cases

LIST OF FIGURES

• Figure 1. System Architecture

• Figure 2. Doubly Linked List Node and Links •

Figure 3. Data Flow Diagram

• Figure 4. Use Case Diagram

• Figure 5. Modular Diagram

• Figure 6. Program Flow Chart

• Figure 7. Sample Console Output

• Figure 8. Project Schedule

LIST OF TABLES

• Table 1. Hardware Requirements •

Table 2. Software Requirements •

Table 3. Node Data Schema

• Table 4. Program Modules

• Table 5. Test Cases

• Table 6. Time Complexity

ABSTRACT

The Music Playlist Manager is a menu-driven application developed in the C programming language to organize and navigate a collection of songs using a doubly linked list. In a basic playlist, users need to add songs, remove songs, view the complete list, and move between tracks in both forward and backward directions. A simple array-based list may require shifting elements after deletion and has a fixed capacity when statically allocated. This project uses dynamically allocated nodes so that the playlist can grow or shrink during execution, subject to available memory. Each node stores a song title and pointers to both the previous and next nodes. These two links make it possible to move to the next song and return to the previous song efficiently. The application provides options to add a song at the end, display the playlist, play or select a song, move to the next or previous song, search by title, delete a song, and exit safely. The program also demonstrates memory management through dynamic allocation and deallocation. The project is intended as an educational implementation of linked-list operations rather than a real audio player; it manages song information and simulated playback navigation, but does not decode or play audio files. The design is modular, with separate functions for common operations, and can be compiled using a standard C compiler. It demonstrates how a data structure can be applied to a familiar real-world task while reinforcing pointers, structures, functions, loops, and memory management.

Keywords: C programming, data structures, doubly linked list, music playlist, dynamic memory allocation, song navigation.

• Clear messages when the playlist is empty or a song is not found.

• Correct next/previous navigation with boundary checks.

• Successful cleanup of all allocated nodes when exiting.

3. LITERATURE SURVEY

Arrays are a simple way to store a group of song titles. They allow direct access by index, but insertion or deletion in the middle can require shifting items. A fixed-size array also places a limit on the number of songs unless a resizing strategy is used.

A singly linked list stores a link to the next node. It supports dynamic insertion and deletion, but backward navigation is not directly available. To return to a previous song, the program may need to traverse from the head.

A doubly linked list stores both previous and next pointers. This supports forward and backward traversal and allows deletion of a known node to be performed in constant time once the node has been located. The cost is additional memory for the previous pointer and extra care when updating links.

For this project, the doubly linked list is a suitable choice because two-way song navigation is a key requirement. The implementation uses dynamic memory allocation and a current-song pointer to represent the selected track.

Approach	Strength	Limitation
Array	Simple and supports indexed access	May require shifting; capacity management needed
Singly linked list	Dynamic size; simple forward traversal	Backward navigation is not direct
Doubly linked list	Forward and backward traversal	Uses extra pointer memory; links must be maintained
Music Playlist Manager	Applies list operations to a familiar task	Console demonstration only; no real audio playback

4. SYSTEM REQUIREMENTS

4.1 Hardware Requirements

Component	Minimum / Recommended requirement
Computer	Desktop or laptop computer
Processor	Any modern or older processor capable of running a C compiler
RAM	2 GB or more recommended
Storage	At least 50 MB free for compiler and source files
Input / Output	Keyboard and monitor

4.2 Software Requirements

Software	Purpose
Operating system	Windows, Linux, or another OS with a C compiler
C compiler	GCC, Clang, or a compatible C compiler
Editor / IDE	VS Code, Code::Blocks, Dev-C++, or a text editor
Language standard	C99 or later recommended
Libraries	Standard C headers: stdio.h, stdlib.h, string.h

The program uses only standard C library functions. No database server, internet connection, or third-party library is required.

5. SYSTEM DESIGN

5.1 System Architecture

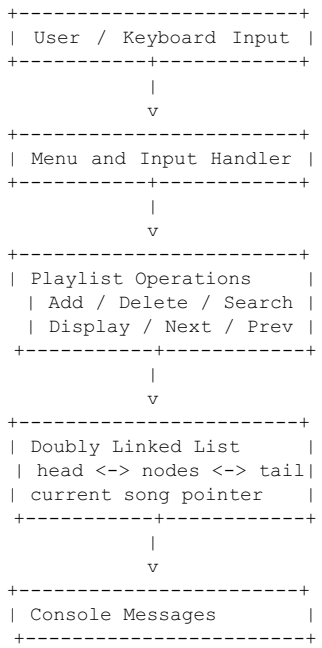


Figure ١. System Architecture

The user selects an operation from the menu. The menu handler validates the choice and calls the relevant function. Playlist functions modify or traverse the doubly linked list, then display a result on the console.

5.2 ER / Data Structure Diagram

This project does not use a relational database, so a conventional database ER diagram is not applicable. The equivalent data-structure relationship is shown below.

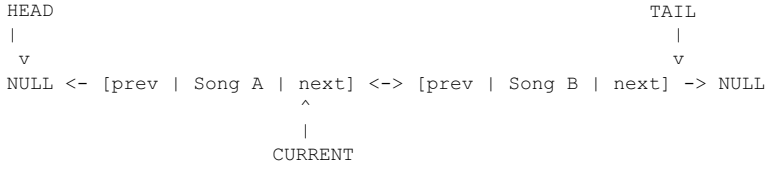


Figure 2. Doubly Linked List Node and Links

Every node contains a song title, a pointer to the previous node, and a pointer to the next node. The head node has prev = NULL and the tail node has next = NULL. The current pointer identifies the selected song when the playlist is not empty.

5.3 Data Flow Diagram (DFD)

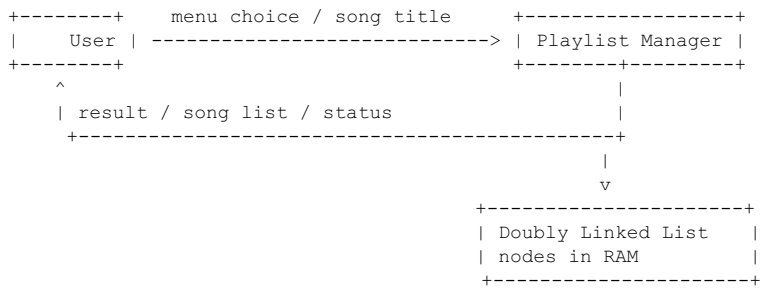


Figure 3. Data Flow Diagram

5.4 Use Case Diagram

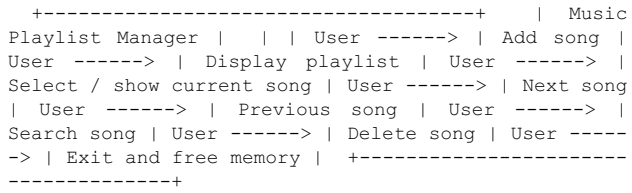


Figure 4. Use Case Diagram

5.5 Data Schema / Node Design

C type

Field		Description
title	char[100]	Stores the song title
prev	struct Song *	Pointer to the previous node
next	struct Song *	Pointer to the next node
head	struct Song *	Pointer to the first node
tail	struct Song *	Pointer to the last node
current	struct Song *	Pointer to the selected song

Node definition: struct Song { char title[100]; struct Song *prev; struct Song *next; }. The list is maintained in memory and has no persistent database schema.

6. IMPLEMENTATION

6.1 Algorithm and Modular Design

Overall algorithm

• Start the program and initialize head, tail, and current to NULL.

• Display the menu and read the user's choice.

• If Add is selected, allocate a new node, read its title, and link it after the tail.

• If Display is selected, traverse from head to tail and print each title.

• If Next or Previous is selected, update current only when the destination node exists.

• If Search is selected, compare the requested title with each node's title.

• If Delete is selected, locate the title, reconnect adjacent nodes, update head/tail/current as needed, and free the removed node.

• Repeat until Exit is selected.

• Free every remaining node and terminate.

Modular diagram

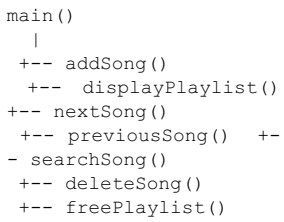


Figure 5. Modular Diagram

Flow chart

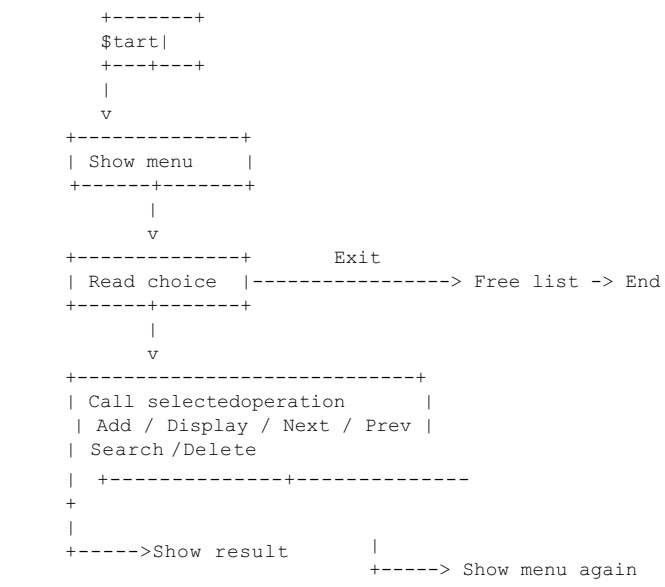


Figure 6. Program Flow Chart

6.2 C Source Code

The following complete program is a console-based demonstration. “Play” means selecting and displaying the current song title; it does not play audio.

```
#include  <stdio.h>
#include <stdlib.h>
#include <string.h>

#define TITLE_LEN 100
```

```
typedef struct Song {
    char  title[TITLE_LEN];
    struct   Song   *prev;
    struct Song *next;
} Song;

Song *head = NULL, *tail = NULL, *current = NULL;

voidreadLine(char *buffer, size_t size) {
    if (fgets(buffer, (int)size, stdin) != NULL) {
        buffer[strcspn(buffer, "\n")] = '\0';
    } else {
        buffer[0] = '\0';
    }
}

Song *findSong(const char *title) {
    Song *temp = head;
    while (temp != NULL) {
        if (strcmp(temp->title, title) == 0)
            return temp;
        temp = temp->next;
    }
    return NULL;
}

void addSong(void) {
    char title[TITLE_LEN];
    Song *node;

    printf("Enter song title: ");
    readLine(title, sizeof(title));
    if (title[0] == '\0') {
        printf("Song title cannot be empty.\n");
        return;
    }

    node=  (Song *)malloc(sizeof(Song));
    if (node== NULL) {
        printf("Memory allocation failed.\n");
        return;
    }

    strcpy(node->title,  title);
    node->prev  =  tail;  node-
    >next = NULL;

    if (head == NULL) {
        head = tail = current = node;
    } else {
        tail->next = node;
        tail = node;
    }
    printf("Added: %s\n", node->title);
}

void displayPlaylist(void) {
    Song *temp = head;
    int number = 1;

    if (head == NULL) {
        printf("Playlist is empty.\n");
        return;
    }

    printf("\n---MUSIC PLAYLIST ---\n");
    while (temp!=NULL) {
        printf("%d.%s", number++, temp->title);
        if (temp==current) printf("    [CURRENT]");
        printf("\n");
        temp = temp->next;
    }
}

void showCurrent(void) {
    if (current == NULL)
        printf("No song selected. Playlist is empty.\n");
    else
        printf("Current song: %s\n", current->title);
}

void nextSong(void) {
```

```
    if (current == NULL) {
         printf("Playlist is empty.\n");
    }  else if (current->next == NULL) {
        printf("Already at the last song.\n");
    } else {
        current = current->next;
        showCurrent();
    }
}

voidpreviousSong(void) {
    if (current == NULL) {
         printf("Playlist is empty.\n");
    }  else if (current->prev == NULL) {
        printf("Already at the first song.\n");
    } else {
        current = current->prev;
        showCurrent();
    }
}

void searchSong(void) {
    char title[TITLE_LEN];
    Song *found;

    printf("Enter title to search: ");
    readLine(title, sizeof(title));
    found = findSong(title);

    if (found == NULL)
        printf("Song not found.\n");
    else
        printf("Song found: %s\n", found->title);
}

void deleteSong(void) {
    char title[TITLE_LEN];
    Song *node;

    printf("Enter title to delete: ");
    readLine(title, sizeof(title));
    node = findSong(title);

    if (node == NULL) {
        printf("Song not found.\n");
        return;
    }

    if (node->prev != NULL)
        node->prev->next = node->next;
    else
        head = node->next;

    if (node->next != NULL)
        node->next->prev = node->prev;
    else
        tail = node->prev;

    if (current == node)
        current = (node->next != NULL) ? node->next : node->prev;

    printf("Deleted: %s\n", node->title);
    free(node);

    if (head == NULL) {
        tail = current = NULL;
    }
}

void freePlaylist(void) {
    Song *temp = head;
    while (temp != NULL) {
        Song *next = temp->next;
        free(temp);
        temp = next;
    }
    head = tail = current = NULL;
}

int main(void) {
    char input[20];
    int choice;
```

```
    do {
        printf("\n===== MUSIC PLAYLIST MANAGER =====\n");
        printf("1. Add song\n");
        printf("2. Display playlist\n");
        printf("3. Show current song\n");
        printf("4. Next song\n");
        printf("5. Previous song\n");
        printf("6. Search song\n");
        printf("7. Delete song\n");
        printf("0. Exit\n");
        printf("Enter choice: ");
        readLine(input, sizeof(input));
        choice = atoi(input);

        switch (choice) {
            case 1: addSong(); break;
            case 2: displayPlaylist(); break;
            case 3: showCurrent(); break;
            case 4: nextSong(); break;
            case 5: previousSong(); break;
            case 6: searchSong(); break;
            case 7: deleteSong(); break;
            case 0: freePlaylist(); printf("Exiting playlist manager.\n"); break;
            default: printf("Invalid choice. Try again.\n");
        }
    } while (choice != 0);

    return 0;
}
```

6.3 SQL Queries, SQL Tables and SQL Normalization

The supplied report template lists SQL queries, SQL tables, and SQL normalization. These items are database-oriented, while the selected project is specifically a C program using an in-memory doubly linked list. The implementation does not connect to SQL and therefore has no SQL queries or relational tables to execute. To address the intent of these headings without falsely claiming a database implementation, the equivalent list operations and node schema are documented here.

Template item	Project-specific equivalent
SQL queries	C functions addSong(), deleteSong(), searchSong(), displayPlaylist()
SQL tables	The Song structure and linked nodes stored in RAM
SQL normalization	Not applicable to the in-memory list; each node stores one song title and two links
Persistent storage	Not implemented; all songs are lost when the program exits

6.4 Program Execution Snapshots

The following are illustrative console transcripts showing expected interaction. They are representations, not captured screenshots from an actual compiler run.

text

```
===== MUSIC PLAYLIST MANAGER =====
1. Add song
2. Display playlist
3. Show current song
4. Next song
5. Previous song
6. Search song
7. Delete song
0. Exit
Enter choice: 1
Enter song title: Perfect
Added: Perfect

Enter choice: 1
Enter song title: Shape of You
Added: Shape of You
```

```
Enter choice: 2
--- MUSIC PLAYLIST ---
1. Perfect  [CURRENT]
2. Shape of You

Enter choice: 4
Current song: Shape of You

Enter choice: 5
Current song: Perfect

Enter choice: 6
Enter title to search: Perfect
Song found: Perfect

Enter choice: 7
Enter title to delete: Shape of You
Deleted: Shape of You

Enter choice: 0
Exiting playlist manager.
```

Figure 7. Sample Console Output

7. RESULTS / OUTPUT

The program provides a menu that allows users to add a song title, display the complete playlist, view the current song, navigate forward and backward, search for a title, delete a song, and exit. Each operation produces a message describing the result. Boundary conditions are handled for empty lists, first and last songs, missing titles, and memory allocation failure.

Test ID	Input / action	Expected result
T1	Display when no songs exist	“Playlist is empty.”
T2	Add “Perfect”	Song is added; current points to it if list was empty
T3	Add “Shape of You”	New node is linked after the tail
T4	Next from first song	Current song becomes the second song
T5	Previous from first song	Boundary message; current unchanged
T6	Search existing title	Song found message
T7	Search missing title	Song not found message
T8	Delete the only song	head, tail, and current become NULL
T9	Delete first or last song	Head/tail updated and adjacent links repaired
T10	Exit	Allocated nodes are freed

Expected result analysis: the playlist maintains the sequence in insertion order. The prev and next links permit two-way navigation. Deleting a node reconnects its neighbours and frees its memory. The code should be compiled and tested in the target environment before final submission; the sample transcript documents expected behavior, not a claim of measured test execution.

Project schedule

Week / phase	Activity
1	Understand the problem and select doubly linked list
2	Design node structure, operations, and menu
3	Implement insertion, traversal, navigation, search, and deletion
4	Test edge cases, correct errors, prepare report

Figure 8. Project Schedule

8. APPLICATIONS

• Learning and demonstrating doubly linked list operations.

• Managing a small ordered collection of song titles.

• Demonstrating next/previous navigation in media-player prototypes.

• Serving as a starting point for a larger playlist application with file storage.

• Teaching C structures, pointers, dynamic memory allocation, and modular programming.

9. ADVANTAGES AND LIMITATIONS

9.1 Advantages

• Supports forward and backward navigation through the playlist.

• Nodes are allocated dynamically, so there is no fixed array capacity.

• Adding at the end is efficient because a tail pointer is maintained.

• Functions are separated by task, making the program easier to understand and maintain. •

Deletion releases memory used by the removed node.

• Uses standard C libraries and can be compiled on many platforms.

9.2 Limitations

• Song information is stored only in RAM and is lost after exit.

• The program manages titles; it does not play audio files.

• Searching and deleting by title require a traversal and take linear time.

• Duplicate song titles are allowed; deletion removes the first matching title.

• Title input is limited to 99 characters plus the terminating null character.

• The console menu has no graphical interface, playlists, metadata, or user accounts.

10. CONCLUSION

The Music Playlist Manager demonstrates how a doubly linked list can be applied to organize and navigate a sequence of songs. The C implementation supports insertion, display, current-song selection, next and previous navigation, searching, deletion, and memory cleanup. Maintaining both previous and next pointers makes two-way movement natural, while dynamic allocation allows the list to grow as songs are added. The project also highlights important programming practices, including checking for empty lists, handling boundary conditions, validating user input, and freeing allocated memory. The current version is an educational console application and does not provide real audio playback or permanent storage. Overall, it meets the data-structure learning objective and provides a foundation for future extensions.

11. FUTURE SCOPE

• Save and load playlists using file handling.

• Add artist, album, duration, and genre fields to each song.

• Allow playlist creation, renaming, and switching between multiple playlists.

• Implement shuffle and repeat modes.

• Connect song records to actual audio files and integrate an audio playback library.

• Add case-insensitive searching and support for duplicate-aware deletion.

• Develop a graphical user interface and improve input validation.

• Add a persistent database only if the project requirements are expanded to include SQL.

12. REFERENCES

• Brian W. Kernighan and Dennis M. Ritchie, The C Programming Language, 2nd Edition, Prentice Hall.

• E. Balagurusamy, Programming in ANSI C, McGraw Hill.

• Reema Thareja, Data Structures Using C, Oxford University Press.

• ISO/IEC 9899, Programming Languages — C (C language standard).

• Course notes and classroom material on linked lists, pointers, structures, and dynamic memory allocation.

13. APPENDIX

A. Complexity Analysis

Operation	Time complexity	Reason
Add song at tail	O(1)	Tail pointer gives direct access to the end
Display playlist	O(n)	Visits every node
Next / Previous	O(1)	Moves through one pointer
Search song	O(n)	May inspect every node
Delete by title	O(n)	Search is linear; relinking after locating is O(1)
Free playlist	O(n)	Frees each node once

B. Important implementation notes

• Always check whether malloc() returns NULL before using the allocated node.

• When adding the first node, set head, tail, and current to that node.

• When deleting the head, update head and set the new head's prev pointer to NULL.

• When deleting the tail, update tail and set the new tail's next pointer to NULL.

• If the current node is deleted, select its next node if available; otherwise select its previous node. •

Free all remaining nodes before program termination to avoid memory leaks.

C. Compilation and execution

```
gcc music_playlist_manager.c -o music_playlist_manager
./music_playlist_manager
```

On Windows with GCC installed, the executable can be run as music_playlist_manager.exe. Save the source code as music_playlist_manager.c before compiling.